Experiment No-3
Array operations and Linear equations

AIM: This lab explores the array operations and linear equation-solving capabilities of MATLAB. By investigating fundamental concepts such as array manipulation and linear algebraic operations, students gain a deeper understanding of MATLAB's computational power and its applications in solving systems of linear equations.

Software Required: Matlab software

Theory:

Array operations

MATLAB has two different types of arithmetic operations: matrix arithmetic operations and array arithmetic operations. We have seen matrix arithmetic operations in the previous lab. Now, we are interested in array operations.

Matrix arithmetic operations

As we mentioned earlier, MATLAB allows arithmetic operations: +, , , and ^ to be carried out on matrices. Thus,

A+B or B+A	is valid if A and B are of the same size
A*B	is valid if A's number of column equals B's number of rows
A^2	is valid if A is square and equals A*A
α*A or A*α	multiplies each element of A by α

Array arithmetic operations

On the other hand, array arithmetic operations or *array operations* for short, are done *element-by-element*. The period character, ., distinguishes the array operations from the matrix operations. However, since the matrix and array operations are the same for addition (+) and subtraction ( ), the character pairs (.+) and (.) are not used. The list of array operators is shown below in Table 3.2. If A and B are two matrices of the same size with elements $\mathbf{A} = [a_{ij}]$ and $\mathbf{B} = [b_{ij}]$, then the command

.*	Element-by-element multiplication
./	Element-by-element division
.^	Element-by-element exponentiation

Table 3.1: Array operators

```
>> C = A.*B
```

produces another matrix C of the same size with elements $c_{ij} = a_{ij}b_{ij}$. For example, using the same 3×3 matrices

```
      1   2   3
A=    4   5   6
      7   8   9
     10   20  30
B=   40   50  60
     70   80  90
```

we have,

```
>> C = A.*B
C       =
10      40      90
160     250     360
490     640     810
```

To raise a scalar to a power, we use for example the command 10^2. If we want the operation to be applied to each element of a matrix, we use .^2. For example, if we want to produce a new matrix whose elements are the square of the elements of the matrix A, we enter

```
>> A.^2
ans     =
1       4       9
16      25      36
49      64      81
```

The relations below summarize the above operations. To simplify, let's consider two vectors U and V with elements U = [ui] and V = [vj].

```
U. * V  produces     [u1v1 u2v2 . . . unvn]
U./V    produces     [u1/v1 u2/v2 . . . un/vn]
 U.^V   produces     [uv1 uv2 . . . uvn ]
```

Operation	**Matrix**	**Array**
Addition	+	+
Subtraction	-	-
Multiplication	*	.*
Division	/	./

Left division	\	.\
Exponentiation	^	.^

Table 3.2: Summary of matrix and array operations

Solving linear equations

One of the problems encountered most frequently in scientiflc computation is the solution of systems of simultaneous linear equations. With matrix notation, a system of simultaneous linear equations is written

$Ax = b$ (3.1)

where there are as many equations as unknown. A is a given square matrix of order n, b is a given column vector of n components, and x is an unknown column vector of n components. In linear algebra we learn that the solution to $Ax = b$ can be written as $x = A_i 1b$, where $A_i 1$ is the inverse of A.

For example, consider the following system of linear equations

$$\{x + 2y + 3z = 1\}$$
$$\{4x + 5y + 6z = 1\}$$
$$\{7x + 8y + 9z = 1\}$$

The coefficient matrix A

$$A = \begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix} \qquad \text{and the vector } B = \begin{matrix} 1 \\ 1 \\ 1 \end{matrix}$$

With matrix notation, a system of simultaneous linear equations is written

$$Ax = b \qquad (3.2)$$

This equation can be solved for x using linear algebra. The result is $x = A^{-1}b$.

There are typically two ways to solve for x in MATLAB:

1. The first one is to use the matrix inverse, inv.

```
>> A = [1 2 3; 4 5 6; 7 8 0];
>> b = [1; 1; 1];
>> x  =  inv(A)*b x        =
```

-1.0000

1.0000

-0.0000

2. The second one is to use the backslash ( )operator. The numerical algorithm behind this operator is computationally efficient. This is a numerically reliable way of solving system of linear equations by using a well-known process of Gaussian elimination.

```
>> A = [1 2 3; 4 5 6; 7 8 0];
>> b = [1; 1; 1];
>> x = A\b
x =
-1.0000
1.0000
-0.0000
```

This problem is at the heart of many problems in scientific computation. Hence it is impor- tant that we know how to solve this type of problem efficiently.

Now, we know how to solve a system of linear equations. In addition to this, we will see some additional details which relate to this particular topic.

Matrix functions

MATLAB provides many matrix functions for various matrix/vector manipulations; see Table 3.3 for some of these functions. Use the online help of MATLAB to find how to use these functions.

det	Determinant
diag	Diagonal matrices and diagonals of a matrix
eig	Eigenvalues and eigenvectors
inv	Matrix inverse
norm	Matrix and vector norms
rank	Number of linearly independent rows or columns

Table 3.3: Matrix functions